System design: House Prices continual-learning service

Project: SYS-304 House Prices **Repository:** LukaBondi/House-Prices **Design snapshot:** October 5, 2026 **Status:** The timed Compose smoke, restart assertion, rollback, and one-time final audit are complete. The live candidate was rejected by the fixed gate and the frozen bootstrap remains selected. The linked logging benchmark is complete. No presentation is claimed. **Markdown review:** October 14, 2026. The Oct 5 measurements and verification results remain historical evidence and were not re-collected for this writing update. The PDF export is generated directly from this Markdown by the builder; it was rebuilt and visually checked on October 20, 2026. The repository URL is provided, but push, course submission, and remote CI execution remain unverified.

Purpose

The service predicts Ames home sale prices from a frozen XGBoost pipeline. The continual-learning extension adds durable request auditing, input-drift monitoring, delayed and provenance-checked labels, scheduled candidate training, and guarded model activation.

The service can decide to keep the incumbent. A candidate is promoted only when it satisfies a fixed policy. A passing feasibility calculation is not an actual serving promotion, and an activation-path fixture is not an accuracy result.

The bootstrap champion is the original fitted model exported to XGBoost UBJ with its fitted preprocessing metadata. The original baseline archive and feature schema remain frozen, while a candidate that passes the gate may replace the active booster version. Native prediction fuses preprocessing into sparse input construction and avoids pandas, scikit-learn, and joblib in the API runtime. Exact-input TTL/LRU caching and request coalescing are enabled. Dynamic batching is available but disabled by default.

Domain, users, and scope

The prediction problem is supervised regression on the Ames housing data: map a home's recorded property features to an estimated sale price. A browser user or API caller supplies available feature values; a successful response returns the estimated `sale_price` in USD, a server-generated `request_id`, and the captured `model_version`. The partial browser form loads sample values from the schema endpoint; omitted values follow the frozen preprocessing behavior. Operations users inspect persisted request health, input distributions, verified-label counts, and trainer outcomes.

The implemented scope is point-estimate serving on one historical dataset. The model has no calibrated uncertainty interval. A changed input distribution can be measured but is not evidence that target error changed. The fixed preprocessing and feature schema constrain what continuation training can learn: new feature definitions or materially new category semantics require a controlled preprocessing/schema migration and full retraining path, which this service does not implement. Real production sale outcomes and a production label-quality contract are also outside the current local simulator.

Conceptual framing and course mapping

The design joins four general ideas: a regression model turns features into a numeric estimate; request telemetry makes the service observable; feature drift compares current inputs with a frozen reference; and delayed outcomes support supervised model evaluation. A model is not deployed merely because new labels exist: a candidate is evaluated on paired examples, stored as an immutable version, and activated only after the fixed gate and serving checks pass. Operational latency, HTTP errors, and feature drift answer different questions from target error, which requires verified outcomes.

The source material for course Lectures 1-3 was not available for this review. This paragraph is a neutral conceptual primer and does not claim to reproduce or verify course-specific lecture coverage.

Functional and quality goals

The project brief calls for logged prediction requests, latency/error and input-distribution monitoring, a real HTTP workload, later verified truth, automatic fine-tuning and evaluation, and automatic deployment of a qualifying candidate. Its trigger alternative is time **or** confidence. The implementation uses a timer; it does not implement a confidence-triggered scheduler.

Class	Goal	Implementation and source	Evidence or boundary
Functional	Accept housing features and return an estimate with request and model identity	API, native serving, browser form	The recorded browser check loaded `Sawyer` and received a $143,950 response; omitted features use fitted preprocessing.
Functional	Persist request lifecycle, prediction, model version, latency, status, and errors	request middleware, SQLite store	Stack smoke records the real HTTP workload and restart persistence. A request row records the prepared outcome, not proof of client receipt.
Functional	Report latency, completed-response errors, and input-distribution drift	monitoring summary, dashboard, frozen reference	Baseline and shift records preserve the Oct 5 windows. Drift is not target-error monitoring.
Functional	Exercise reproducible traffic against the real HTTP API	workload generator	Scenario definitions and eligibility are described below; the actual 40-label path is in the smoke evidence.

Class	Goal	Implementation and source	Evidence or boundary
Functional	Use only later, provenance-checked outcomes for continuation training	truth importer, trainer	The live run imported 40 delayed labels. The importer rejects changed, failed, ineligible, or not-yet-available observations.
Functional	Train and evaluate candidates, then activate only a qualifying candidate	trainer, registry, activation API	The actual candidate was rejected. A separately tagged same-model fixture checks the activation route only; see run evidence and rollback evidence.
Functional	Preserve data/model boundaries and provide a repeatable deployment	Compose topology, CI workflow definition	Linux Compose smoke was recorded Oct 5. The workflow exists in the repository; remote CI execution and submission status were not verified for this review.
Nonfunctional	Keep request logging bounded and expose acknowledgement failures	writer and logger, SQLite store	Queue, busy-timeout, and acknowledgement limits are documented below. A failed acknowledgement does not become an unlogged success response.
Nonfunctional	Protect model/pointer consistency and private administration	activation API, proxy rules, Compose	Token, loopback, and volume ownership are described below; this is not an internet-facing security boundary.
Nonfunctional	Make performance observable without inventing an SLO	logging benchmark, monitoring code	The benchmark is a local Windows comparison, not a capacity or service-level guarantee. The project has no stated numerical latency or availability SLO.

These are implemented project goals rather than a claim that every objective was achieved in the actual run: automatic activation is conditional on all checks; the measured real candidate did not qualify.

Design alternatives and trade-offs

Decision	Alternative	Reason for the current choice and its cost
SQLite WAL on one host	PostgreSQL or a managed database	SQLite keeps the demo self-contained and provides durable local transactions. Its single-writer behavior and shared file become the first scaling concern; read-only dashboard queries and trainer work still share that store.
Small static dashboard with domain-specific summary	General observability/metrics platform	The custom page can show request lifecycle, labels, runs, versions, and drift without a frontend toolchain. It has no alerting, multi-tenant access, or independent metrics-store isolation.
Time-triggered scheduler	Confidence-triggered training	A timer is deterministic and directly satisfies the brief's time-or-confidence option. It can wake when no useful data exists, so minimum-label, snapshot, backoff, and cooldown checks gate actual work. Confidence-based triggering is not implemented.
One API worker with process-local cache/coalescing	Multiple workers or replicas	A single process keeps cache and in-flight coalescing coherent and simplifies the pointer transition. It limits horizontal throughput; adding workers alone will not remove the SQLite write bottleneck.
Delayed, verified labels and fixed preprocessing	Accept caller-provided targets or immediately train on predictions	Keeping targets separate and checking source membership, hash, features, completion, and delay protects label provenance. It slows learning and cannot add new features/encodings without a separate schema migration/full retrain.
Native UBJ inference and fitted preprocessing	Load the original Python pipeline at request time	Native inference reduces serving dependencies and preserves the frozen transform contract. Its hand-built mapping must remain parity-checked against the reference pipeline.

Data boundaries and evidence

The source CSV is identified by SHA-256:

1e18addf81e5e4d347cc17ee6075bbe4a42b7fa26b9e5b063e8f692a5f929d41

The hash-pinned split has disjoint partitions:

Partition	Rows	Use
Original fit	1,168	Existing fitted model and immutable input-drift reference
Adaptation	146	Unchanged, successful HTTP examples that may receive delayed truth
Promotion gate	73	Paired champion/challenger evaluation only
Final audit	73	One descriptive evaluation after model selection
Kaggle test	1,459	Unlabeled traffic and parity checks

These rows came from the original train/test split. The historical 292-row holdout was already evaluated in prior project work, so the new gate and audit are reserved from continual-learning fitting but are not statistically untouched data. Repeated use of the small gate limits the strength of any accuracy conclusion.

Only a verified adaptation row can receive a label. The source CSV hash is checked before the target is read. The importer then verifies adaptation membership, exact canonical feature equality/fingerprint, persisted successful prediction, request/run provenance, completion time, and label availability after the configured delay. Repeated HTTP calls remain independently audited but cannot create duplicate training observations. Changed, synthetic, unlabeled, fit, gate, audit, failed, and post-cutoff observations are ineligible. A model prediction is never ground truth.

The frozen native artifact SHA-256 is:

4e605ea450b01531142398e273729f326df625ae21b8f0a5235f030382721db4

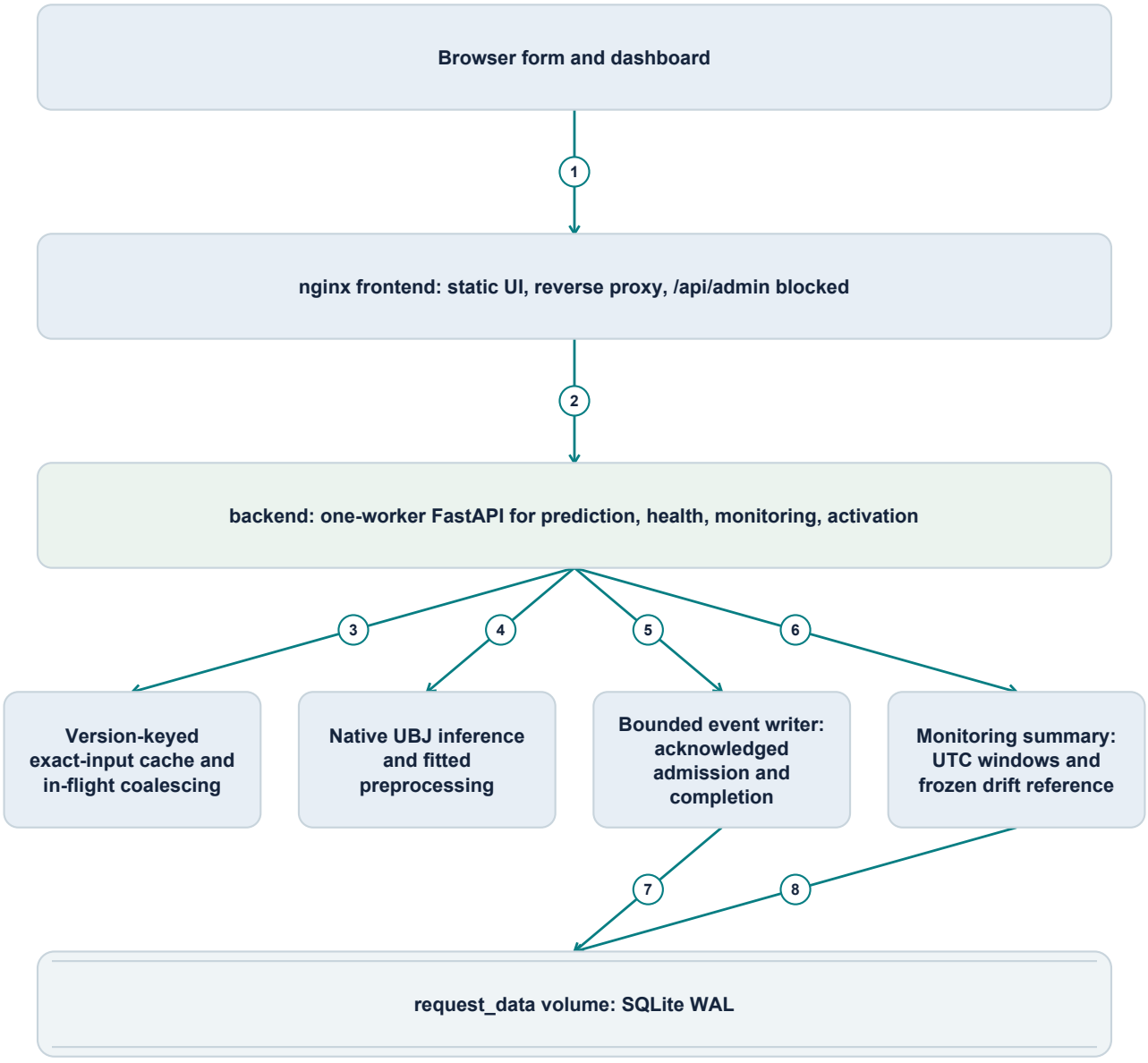
The parity exporter checks all 1,460 original training rows, all 1,459 test rows, and three edge records. Recorded maximum absolute prediction difference: zero. The frozen artifact was preserved.

Serving and monitoring architecture

These two component views show service boundaries, storage ownership, and their main data and control flows. The numbered request lifecycle below then expands a single prediction request.

Serving and monitoring components

Serving and monitoring components

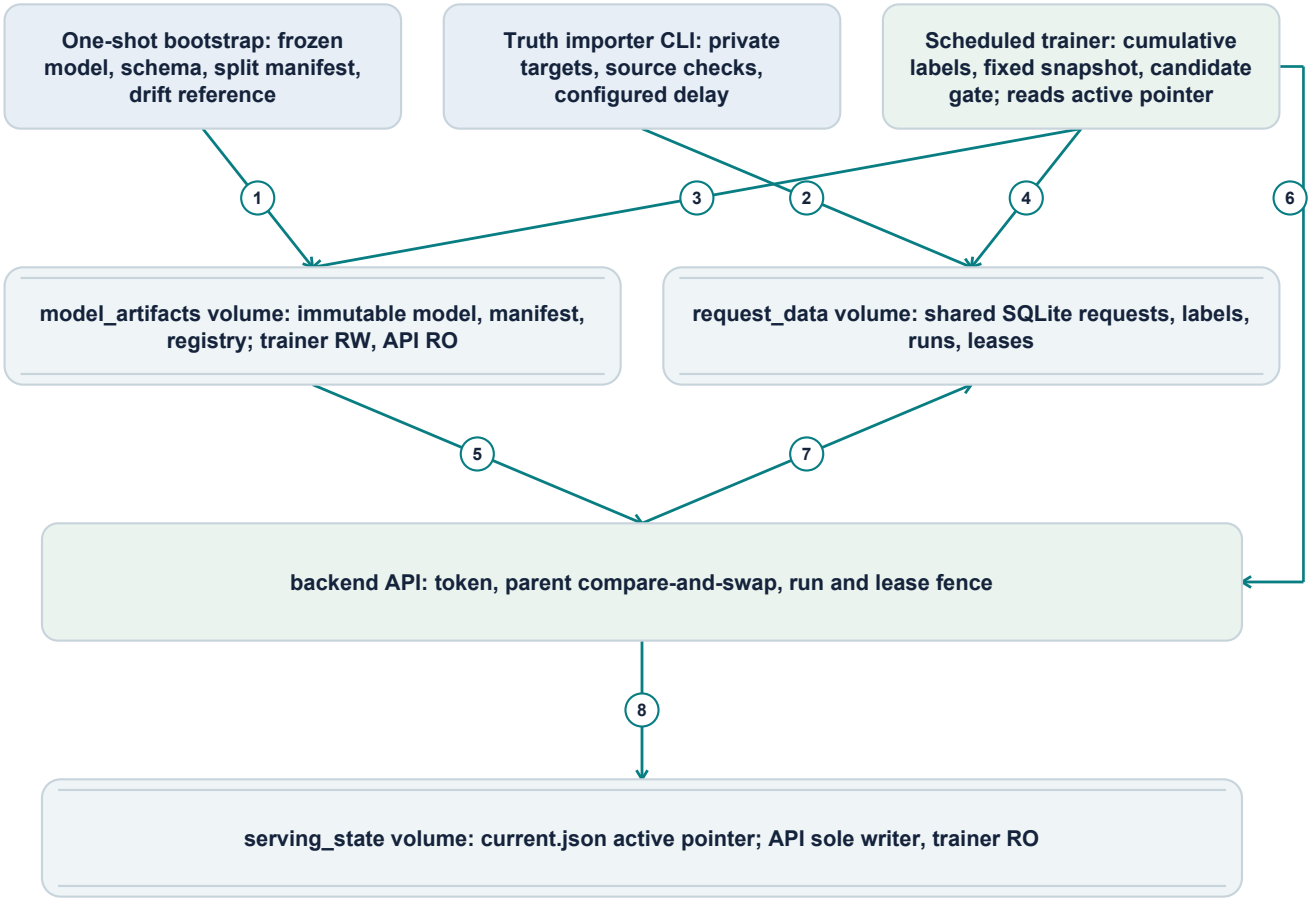


Edge key

1. form, dashboard, prediction
2. reverse-proxied API calls
3. versioned lookup
4. miss: native inference
5. audit event
6. dashboard poll
7. write events
8. read summary

Learning, registry, and serving-state components

Learning, registry, and serving-state components



Edge key

1. seed baseline
2. verified labels
3. candidate write
4. read labels; write run/lease
5. API read-only mount
6. token activation
7. audit/fence
8. pointer write

Request lifecycle - vertical rendering of the source sequence

1. Browser or API client -> nginx: POST /api/predict
2. nginx -> FastAPI one-worker process: POST /predict
3. FastAPI one-worker process: Assign request UUID and capture bounded context
4. FastAPI one-worker process -> Bounded event writer: Admit pending request
5. Bounded event writer -> SQLite WAL: Insert admission row
6. SQLite WAL -> Bounded event writer: Commit
7. Bounded event writer -> FastAPI one-worker process: Admission acknowledged
8. FastAPI one-worker process: Capture immutable model, schema, and version snapshot
9. FastAPI one-worker process -> Version-qualified cache: Lookup features under captured version

Cache hit or in-flight coalescing

Version-qualified cache -> FastAPI one-worker process: Cached or shared prediction

Cache miss

FastAPI one-worker process -> Native UBJ inference: Predict with captured snapshot

Native UBJ inference -> FastAPI one-worker process: Prediction

10. FastAPI one-worker process -> Bounded event writer: Complete with prepared status, prediction, version, and timings
11. Bounded event writer -> SQLite WAL: Update request lifecycle and metrics
12. SQLite WAL -> Bounded event writer: Commit
13. Bounded event writer -> FastAPI one-worker process: Completion acknowledged
14. FastAPI one-worker process -> nginx: Prepared response and X-Request-ID
15. nginx -> Browser or API client: HTTP response

If admission is not acknowledged, work is rejected. If completion cannot be acknowledged, the API does not claim a logged success.

Every HTTP route receives a server-generated request UUID and an audit event. Prediction bodies are capped at 65,536 bytes. Malformed and oversized requests retain bounded parse context; oversized bodies are marked truncated. Admission

is durably acknowledged before prediction work and completion is durably acknowledged before a prepared response is returned. The event writer has a bounded queue and acknowledgement budget. Logging failures are surfaced in health/monitoring and produce a bounded 503 when an acknowledgement deadline cannot be met.

An event status code records the prepared HTTP outcome. It does not prove that bytes reached the client. A process crash can leave an admitted row pending. If a completion write is uncertain, the API does not claim that an unlogged success was delivered. Logging-failed outcomes remain visible when persistence permits, and they are shown as a subset of completed responses rather than counted twice.

The monitoring summary draws a coherent UTC-bounded window from SQLite. Its panels include prediction totals/rate, completed/pending/aborted counts, handler latency, completed-response 4xx/5xx rates, cache hits/misses/coalescing, logger health/failures, active model version, delayed labels, retraining outcome, and input drift. Health checks, dashboard polling, and admin requests are audited but do not pollute prediction metrics. Missing numeric fields remain unavailable. Empty denominators yield null rates or quantiles.

The static dashboard polls every five seconds and offers 15-, 60-, and 1,440-minute windows. It validates the response envelope, distinguishes a missing retraining field from explicit last_run null, shows nested champion/challenger RMSLE and error/rejection reasons, and exposes actual UTC start/end bounds. Fetches are time-limited. A generation guard discards stale successes and errors after a window change and queues a refresh for the latest selection.

The drift reference is computed once from the original fit partition, excluding row identity and target. It stores fixed per-feature distributions with explicit missing, numeric underflow/overflow, and categorical unknown buckets. Monitoring computes base-2 Jensen-Shannon distance, the square root of divergence. At least 100 valid prediction observations are required before an advisory flag; the default distance threshold is 0.10. Drift means observed inputs differ from the reference. It does not measure sale-price error or establish that a model is inaccurate.

Form coverage and missing values

The browser form has 12 visible fields for the 79-feature model. **Load Sample** fetches `schema.sample` but copies values only for those fields; it does not fill the other schema features. Even with all controls filled, 67 features are absent from the browser payload, including `GarageArea` and `BedroomAbvGr`. API canonicalization adds omitted, empty, or null fields as Python `None`, serialized as JSON `null` in the stored full-schema record. Monitoring reads this pre-imputation record; inference separately applies frozen fitted medians for numeric features and most-frequent modes for categorical features. A missing numeric value is not zero; a provided numeric `0` is a real value. An unfamiliar non-empty category is not missing: the one-hot encoder ignores values outside its vocabulary while monitoring counts them as unknown. Invalid types and non-finite numeric values can still fail validation.

Across N fully filled sample-form calls, each omitted feature has observed_count=N and missing_count=N. Each accepted canonical record contributes one observation per feature; repeating identical values creates one distinct input:

Feature across ten identical sample requests	Observed	Missing	Non-missing	Distinct inputs
`GarageArea` (omitted by the form)	10	10	0	1
`OverallQual` (provided by the form)	10	0	10	1

`observed_count` includes the missing bucket, so `nonmissing_count = observed_count - missing_count`. Distinct inputs are based on canonical feature fingerprints across requests; they describe variety, not the number of rows or predictions.

Input drift: interpretation and limits

The fixed reference represents 1,168 original training-fit rows, excluding `SalePrice` and `Id`. Buckets represent missing values, numeric ranges and underflow/overflow, known categories, and unfamiliar categories. Each row aggregates one feature across the window; it is not one prediction or whole-model accuracy. The dashboard sorts all 79 features by distance but shows only the top ten, rounded to two decimal places.

Base-2 Jensen-Shannon distance ranges from 0 to 1. If the reference is fully present but incoming values are all missing, the distributions are disjoint and distance is 1; missing mass in the reference usually lowers it. A value near 1 can round to `1.00`. The current form omissions can dominate the table even when all visible fields are filled. High distance indicates input shift, not 10% error, 100% wrong predictions, sale-price error, or proof of inaccuracy.

The 100-valid-input minimum gates only the aggregate advisory; per-feature rows still appear before it. Once met, any feature distance at or above the default `0.10` threshold can trigger the advisory. This is a distance threshold, not a request percentage or accuracy score. Drift uses accepted canonical inputs, not successful HTTP responses; inference or logging can fail afterward. Full-feature, varied workloads make population changes easier to interpret. Cohort-aware references and "not collected" markers are future proposals, not implemented; read current distances and advisories as emitted.

Reading the windowed monitoring panels

The dashboard queries 15-, 60-, or 1,440-minute windows. Request rows and drift observations are filtered by `received_at` in the selected UTC interval; labels are filtered by `available_at`. The window start and end shown in the UI are the actual UTC bounds. A count of zero is a real empty count. Rates and latency percentiles with an empty denominator are null and render as unavailable. Fewer rows in a later short-window view can reflect normal expiry from that interval, not data loss.

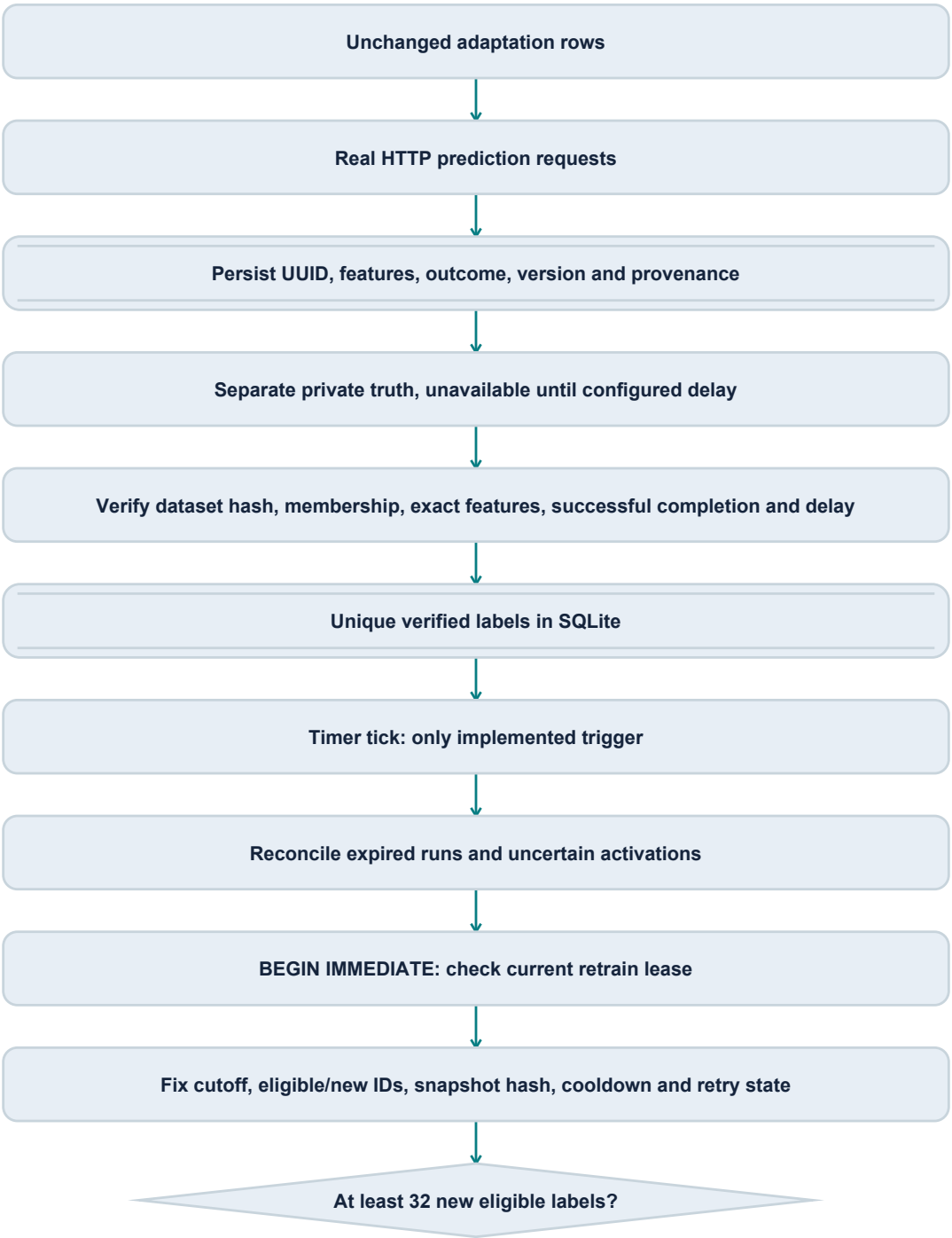
Panel	What the value means	Scope or caveat
Requests and rate	One row per `POST /predict` request; rate is the window count divided by window minutes.	It is a window average, not an instantaneous rate. Other API routes are audited but excluded from these prediction panels.
Completed, pending, aborted	Completed includes prepared HTTP success and error responses; pending is still incomplete; aborted records an interruption.	Pending does not imply eventual failure or success. A prepared response does not prove the client received its bytes.
4xx/5xx and latency	Error rates use completed responses as their denominator; p50/p95 are persisted handler duration. p50 is the median (half of measured requests are at or below it); p95 is the value at or below which 95% fall.	Pending and aborted rows are excluded from the error-rate denominator. Handler duration includes admission wait, but excludes completion-commit wait and network delivery. These metrics describe serving behavior, not target accuracy.
Cache	A hit reuses a version-qualified result; a miss performs inference; coalesced requests share in-flight work.	Each HTTP request still has its own request UUID and audit row, including cache hits and coalesced requests.
Drift	Per-feature base-2 distance, observed/missing/unknown counts, advisory state, and distinct-input count.	Each row aggregates one feature over the window, not one prediction or whole-model accuracy; only ten of 79 features are shown. Missing contributes to observed. Do not sum feature missing counts as failed requests.
Active model and latest run	The currently active model version and the latest recorded training outcome.	Active version is current state; latest training run is global and is not filtered to the selected time window.
Labels	Labels whose truth became available during the selected window, with distinct observation identities.	This is not the total labels in the database or the number of prediction targets. Retraining eligibility uses its own cumulative eligible-data snapshot and cutoff; the dashboard window alone cannot establish the 32-new-label trigger.
Logger health	Whether logging is enabled, its current health, and process-local failure diagnostics.	Enabled does not mean healthy. Failure counts and write-wait totals are process-local, reset on process restart, and are not windowed; persisted request rows remain in the named volume.

Every HTTP API route is audited, while nginx-served static files are not API requests. The dashboard polls `/health` and `/monitoring/summary` every five seconds. Those polls are excluded from prediction totals and drift, but each still creates audited API traffic and database writes. Cache hits also retain the request's admission and completion writes, so adding dashboard clients or API replicas does not remove SQLite write pressure. A logging-queue or database outage can prevent some arrivals from being persisted; the API returns a bounded 503 when it cannot acknowledge admission or completion, and process-local logger diagnostics can report failure even when no audit row becomes durable. `logging_failed` is a subset of completed responses, not an extra count; 5xx responses remain in completed, while pending and aborted requests are not counted as completed errors.

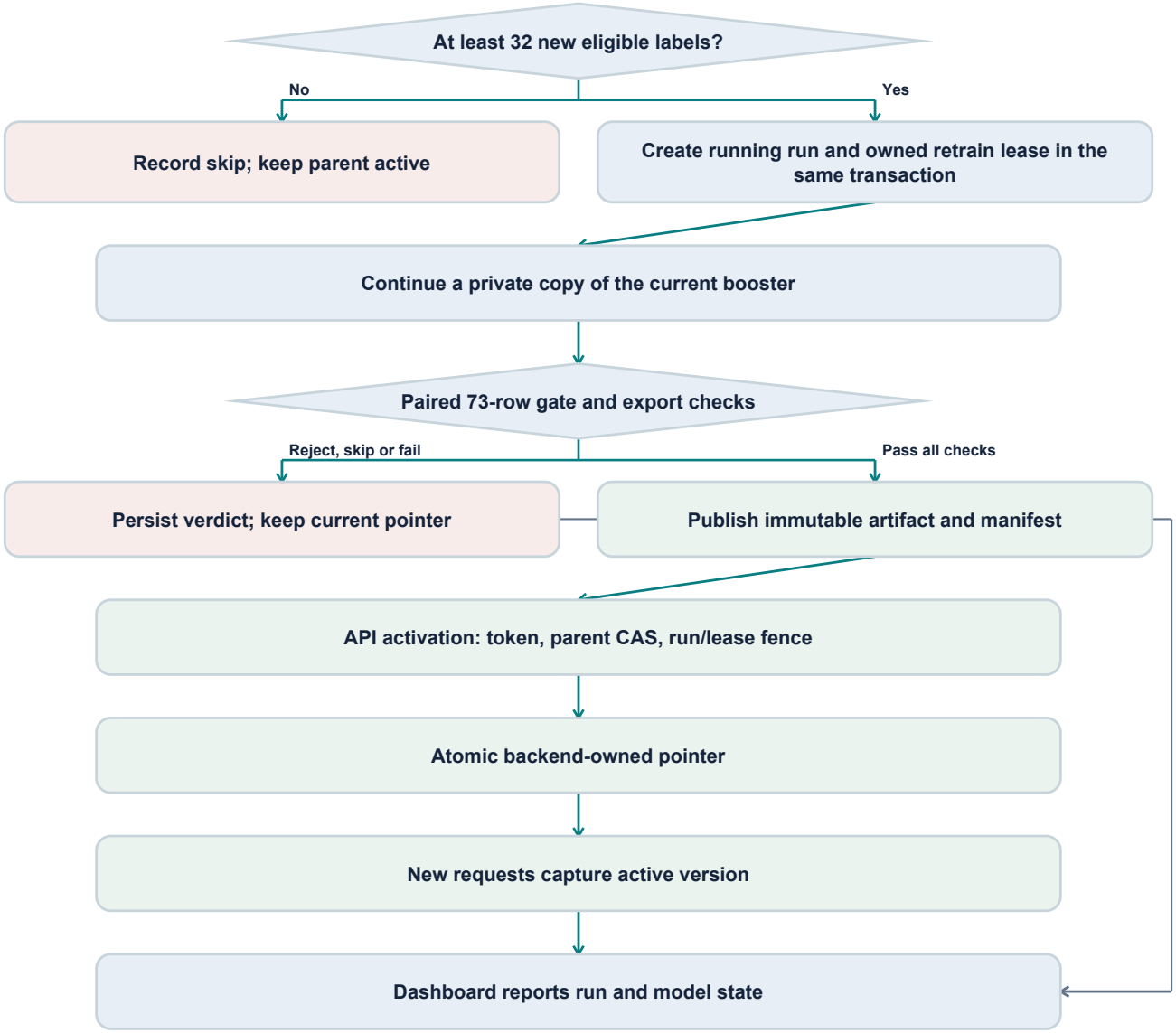
An empty window can show real zero counts while rates and latency percentiles are unavailable because their denominators are empty. On API failure, the monitoring-unavailable message and last-success time identify stale data, not a live reading.

Request-to-training-to-serving flow

Training flow - part 1 of 2, labels through timer decision



Training flow - part 2 of 2, gate outcome through serving pointer



The scheduler is time-triggered, with a default Compose interval of 3,600 seconds and a short configurable interval for controlled demos. It requires 32 new verified labels. Each run fixes a cutoff and snapshot hash, records new and cumulative observation IDs, and uses an owned trainer lease to prevent a stale process from publishing.

The candidate continues a private copy of the current booster using the original preprocessing coordinates, base score, objective, and seed. The `32 new labels` requirement is the run trigger, not the total fit set: each continuation replays the 1,168 original fit rows plus all cumulative eligible adaptation labels available at that run's fixed cutoff. Replaying the original rows helps retain baseline coverage but does not guarantee against forgetting. The declared update adds 25 rounds at learning rate 0.01 and respects a cumulative tree-growth cap of 600 by default. It does not search hyperparameters against the gate.

Gate condition	Policy
Candidate output	All finite and nonnegative
RMSLE	At least 1% relative improvement over the champion
RMSE	No worse than the champion
Export/serve validation	Native parity, manifest/hash/schema checks, staged load, representative edge predictions

Gate condition	Policy
Outcome	Reject on any failed condition; keep the current version active

A rejected snapshot is not retrained every timer tick. Failed runs use snapshot-specific retry backoff. A rejection, skip, or failure is legitimate. The code does not fabricate a real trainer run or lease for a controlled activation fixture.

Metric definitions and project policies

For nonnegative actual prices `y_i` and predictions `ŷ_i`, MAE is the mean absolute dollar error `sum(|ŷ_i - y_i|) / n`; RMSE is `sqrt(sum((ŷ_i - y_i)^2) / n)` in dollars; RMSLE is `sqrt(sum((ln(1 + ŷ_i) - ln(1 + y_i))^2) / n)`, a unitless measure that gives more weight to relative differences. None of these values is a confidence score.

The implemented relative RMSLE gain is `(champion RMSLE - candidate RMSLE) / champion RMSLE`. The candidate must reach at least `0.01` (1%) and its RMSE must be no greater than the champion's RMSE on the same gate rows. A zero champion RMSLE yields zero gain and cannot qualify. Candidates must also return finite, nonnegative predictions and pass export, staged-load, schema, preprocessing, and serving-parity checks. The implemented native-parity tolerance is `rtol=1e-6` and `atol=1e-4`, plus equality after rounding to cents; the recorded parity runs had maximum absolute prediction difference zero.

These values are project policies, not universal model-quality rules or a claim from a course lecture: minimum 32 new verified labels per candidate; timer-only scheduling (Compose default 3,600 seconds); 25 added boosting rounds at learning rate 0.01; default cumulative tree cap 600; and an advisory drift threshold of 0.10 after 100 valid prediction observations. Feature distances and missing/unknown counts can still be shown below 100 observations; only the aggregate advisory flag is withheld until the minimum sample is met. No confidence-triggered trainer, calibrated model-confidence output, automatic canary, or automatic model-error rollback is implemented.

Workload scenarios and label eligibility

The workload generator issues actual concurrent HTTP requests and writes a run manifest; where appropriate it writes targets to a separate private truth manifest. The scenarios serve distinct purposes:

Scenario	Input source and behavior	Can produce training labels?
`baseline`	Original fit-partition examples sent through `/predict`; useful for ordinary request and baseline monitoring traffic.	No. Fit-partition membership is outside the adaptation partition.
`repeated`	Repeats a small sample of unchanged adaptation rows to exercise cache/coalescing and independent request audits.	At most one observation per dataset hash/source ID; repeats never create duplicate labels.
`adaptation`	Sends distinct, unchanged rows from the fixed 146-row adaptation partition and records their private target separately.	Yes, only after the importer validates the request and delayed truth.
`drift`	Replays adaptation IDs with controlled numeric shifts, unknown categories, and/or missing values; no observation claim or truth is attached.	No. Altered traffic is monitoring data, not labeled training data.
`malformed`	Sends invalid JSON and invalid feature-envelope/type cases to exercise bounded error handling.	No.
`unlabeled`	Sends feature-only records from `test.csv`, which is checked to contain no `SalePrice`.	No.

The API records observation metadata as an untrusted claim. Only the private trainer-side importer can insert a label: it checks the pinned source hash before reading targets, adaptation membership, canonical feature equality/fingerprint, a matching persisted successful completed prediction, run/request provenance, and that truth availability satisfies the configured delay. The database also enforces one label for each `(dataset_sha256, source_id)` pair. Repeated requests can therefore remain separate audited calls without multiplying the training observation. Prediction output, synthetic drift, malformed input, an unsuccessful or pending request, caller-edited features, and an early target cannot serve as ground truth.

Immutable models and safe activation

The bootstrap process publishes the frozen native artifact as an immutable registry version. The API initializes or validates the active pointer from this trusted baseline when necessary. The registry stores each immutable model archive and manifest; Compose gives the API read-only artifact access. A scheduled trainer can add versions but cannot modify the backend-owned serving pointer.

For a real candidate, training writes an immutable archive and manifest, then asks the API to activate it. The request carries a candidate version, expected parent, idempotency activation ID, and trainer-run ID. The API validates the artifact, candidate gate, and parent. Under an activation mutex and short database write transaction, it verifies the live matching trainer lease and snapshot/run linkage, compares the current parent, atomically replaces current.json, and completes the activation audit. The scheduled trainer makes this activation request automatically after every gate and export check passes; no human approval step is part of the current flow. The API alone publishes the pointer. The trainer reports promotion only after the API confirms committed status and active version. A later rollback remains an explicit operator action.

Predictions capture one immutable model/schema/version snapshot before validation. The captured version is part of cache/coalescing keys. Work already in progress finishes on its captured snapshot, even if the serving pointer changes. On restart, the API validates the pointer and reconciles pending activation records. An activation whose outcome remains unknown blocks new trainer runs until it is reconciled. Lease heartbeats use a fresh clock reading under the database update lock. A repeated lookup for an already committed activation returns that transaction's historical committed result even if a later activation changed the pointer; health or current.json supplies the current serving version. Rollback targets the previously served version or the frozen bootstrap and uses expected-current comparison through the protected backend route.

The smoke tool can exercise the promotion route with an explicitly marked same-model fixture if the real timed candidate is rejected. The fixture uses frozen bytes under a distinct version ID. It has no trainer run ID or lease and supports only an activation-path claim; it says nothing about model quality.

Deployment and scaling

The Compose stack contains a one-shot trainer-image bootstrap, one API, one scheduled trainer, and nginx. Named volumes separate the SQLite database/WAL, immutable model registry, and serving-state pointer. The backend gates frontend/trainer startup on health. Ports bind to loopback. Nginx serves the static form and dashboard and explicitly denies public access to admin paths. The shared admin token is passed to trusted API/trainer processes only. Normal Compose shutdown preserves named volumes; explicit volume removal is a full reset.

State and storage ownership

Compose state	Location and writer	Contents and lifetime
`request_data`	`/data`; API event writer plus trainer importer/scheduler	`requests.sqlite3` in WAL mode holds request lifecycle and metrics, verified labels, run verdicts, activation intents/results, and the singleton trainer lease. A named volume survives ordinary `docker compose down`; removing volumes deletes it.
`model_artifacts`	`/artifacts`; bootstrap and trainer write immutable files, API mounts read-only	Frozen bootstrap and schema/reference metadata, candidate archives, and manifests. New candidates use new version directories; the API does not mutate candidate files.
`serving_state`	`/state`; API writes `current.json`, trainer mounts read-only	Backend-owned active-version pointer. The trainer proposes an activation to the API but cannot publish this file.
Frontend image	Static files baked into nginx image	Form, dashboard, and proxy rules; no persistent business state.

Outside Compose, the API and trainer default their local data paths under the user's application-state directory (`LOCALAPPDATA` on Windows, with an OS temporary-directory fallback); `DATA_DIR`, `DB_PATH`, registry, and serving-state environment variables can override these locations. This keeps local state out of the synced source checkout.

The database separates entities rather than treating each timer run as an unstructured log. `requests` uses the server UUID as its key and stores received/completed times, HTTP lifecycle/status, bounded body and parsed features/fingerprint, prediction, captured version, cache result, source/run provenance, terminal reason, logging-failure flag, and persisted

handler duration. The event logger separately exposes process-local admission/completion wait totals and counts; those are not persisted per-request timing fields. `labels` uses the deterministic observation ID `dataset_sha256:source_id` (the associated `request_id` remains a UUID), plus dataset/source identity, canonical features and fingerprint, sale price, and availability time; `(dataset_sha256, source_id)` is unique. `runs` stores parent version, fixed snapshot hash, status, times, metrics, and details. `activations` stores an idempotency ID, expected parent, candidate, status, and details. `trainer_leases` stores the `retrain` owner, expiry, and snapshot hash. Indexed time, model-version, observation, fingerprint, label-availability, run, activation, and lease fields support bounded reads and ownership checks.

The store has a 256-command writer queue, a 100 ms SQLite busy timeout, and a 250 ms default write-acknowledgement budget. There is no automatic retention or backup/restore guarantee in this design. A bounded archive helper can copy and remove only old terminal request rows without dataset/source provenance; it does not archive labels, provenance-bearing requests, or pending work and is not an automatic retention policy. Named-volume persistence protects against ordinary container recreation, not storage loss, and is not high availability. If `request_data` is lost or corrupt, bootstrap cannot recreate request, label, run, or activation history; that history requires a valid operator backup. If `serving_state` is lost, startup may reinitialize the pointer to the frozen baseline. If `model_artifacts` is lost, candidate archives require a valid backup. Backup restoration was not tested. Operators must arrange and test backups and retention before production use.

Security and trust boundaries

The API binds its host port to loopback, and the frontend binds its port to loopback. Nginx serves static files and proxies `/api/` calls while returning 404 for `/api/admin` and `/api/admin/` routes. Protected activation, rollback, and activation-status calls require `Authorization: Bearer <ADMIN_TOKEN>`; Compose passes this secret only to the API and trusted trainer. Compose includes a conspicuous `local-demo-only-change-this-value` fallback, which is suitable only for an isolated local demo. Set a private `ADMIN_TOKEN` in the shell or an ignored `.env` file before running in any shared environment; `.env` is excluded by the repository ignore rules, and secrets must not be committed. Rotate it by updating the environment and recreating both trusted containers together so they receive the new value.

Prediction-body capture is capped at 65,536 bytes; malformed and oversized bodies keep only bounded parse context. These boundaries do not provide TLS, user accounts, per-user authorization, rate limiting, encrypted-at-rest storage, or internet-facing protection. Keep this deployment private to a trusted local network unless a separate production perimeter and identity layer is added.

The initial system is deliberately single-host:

- SQLite WAL has one serialized writer; request persistence and retraining share the same database.
- One Uvicorn process keeps the process-local TTL cache and in-flight coalescing coherent.
- The trainer runs separately but competes for host CPU and database I/O.
- The immutable registry and current pointer are local Docker volumes.
- The dashboard queries bounded indexed windows, while event history grows until explicitly archived.
- Truth is a delayed local demonstration input; production label integration and label-quality checks need stronger source contracts.

The 100x request-volume target is a planning prompt, not measured capacity. A staged response is:

1. **Contain and measure.** Keep the current loopback deployment private. Establish a service objective with stakeholders rather than inventing a threshold here. Replay the real expected request mix on target hardware and collect client and handler latency, completed-response errors, queue depth/ack failures, SQLite lock time, disk/WAL growth, CPU/memory, cache behavior, and trainer contention. The Oct 5 logging benchmark is a local Windows comparison only. Health, dashboard, and admin calls are audited even though they are excluded from prediction counts; more dashboard pollers therefore add database writes. A cache hit also still pays for admission and completion acknowledgements. More API replicas alone cannot remove a shared SQLite writer bottleneck.
2. **Stabilize data operations.** Define retention and tested backup/restore, checkpoint and monitor WAL growth, and bound dashboard query cost. If measurements show write contention or multi-host needs, migrate the event store to PostgreSQL and a durable ingestion/outbox path while preserving request UUID idempotency and the rule that admission is acknowledged before inference and completion before response.

3. **Scale prediction serving.** Only after the shared persistence path is safe, add API replicas behind a load balancer and enforce bounded request sizes, concurrency, timeouts, and back-pressure. Choose a version-keyed shared cache/coalescing service or explicitly measure the duplicate work and memory cost of replica-local caches. Keep each request pinned to its captured model version during activation.
4. **Isolate learning and artifacts.** Move training to queued workers with explicit CPU/memory quotas, retain one fenced owner for each candidate snapshot, and store immutable archives/manifests in remote object storage. Replace the local pointer with a durable versioned control plane that preserves parent compare-and-swap, idempotent activation, recovery of uncertain outcomes, and a separately authorized rollback path.
5. **Improve release evidence.** Use fresh temporal or external labels, measure target error and relevant segments, add shadow/canary evaluation, and define a human response for degraded quality before broad rollout. Feature drift can prioritize investigation, but it cannot itself establish target error. A future confidence-based trigger would first need calibrated uncertainty; the current XGBoost output is not calibrated confidence.

These are staged design proposals, not implementation claims, deployment commitments, or a measured 100x result. Continual-learning opportunities are to integrate verified real sale outcomes, monitor label freshness and quality, evaluate on temporally fresh holdouts, compare target error by relevant property segments, and test active-label selection or confidence-based scheduling only after the signal is calibrated and validated. Any new feature set or encoding still needs an explicit schema migration or full retraining workflow.

Current constraint	Implemented protection	100x direction
SQLite write serialization and history growth	Bounded acknowledged writer, health visibility, explicit archive/checkpoint helpers	PostgreSQL plus durable event ingestion and retention
Single API process and process-local cache	One captured model version per request; bounded cache and queue behavior	Multiple API replicas with coordinated rollout and measured shared/local cache
Training competes for CPU and shared database	Separate trainer container, singleton lease, fixed snapshot cutoff and tree cap	Isolated training workers, resource quotas, queued jobs and object-backed immutable registry
Sparse, delayed, or incorrect labels	Hash-pinned source rows, private truth import, delay/cutoff and minimum count	Fresh temporal labels, label-quality monitoring, stronger source integration
Small, historically evaluated promotion gate	Fixed paired RMSLE/RMSE policy and one reserved descriptive audit	Fresh temporal/external holdout, canary/shadow evaluation
Lost activation acknowledgements or pointer/database split	Idempotency ID, parent CAS, atomic API-owned pointer and restart reconciliation	Replicated control plane, canary rollout and automated rollback
Repeated dashboard queries on event history	Indexed bounded windows and small static dashboard	Materialized summaries and separate monitoring storage

Failure behavior and safe recovery

Failure	Current protection and observable result	Safe recovery sequence
SQLite unavailable/full, locked, or the bounded writer queue/acknowledgement budget is exhausted	Admission failure returns 503 before prediction work. If completion acknowledgement fails, the middleware does not report a logged success and attempts a `logging_failed` terminal update where storage permits.	Stop adding workload; preserve the volumes; restore disk/database availability and check logger health/queue diagnostics. Resume only after admission and completion acknowledgements work. Keep failed, pending, and logging-failed rows out of the label importer; do not fabricate a success row.
API stops after admission or during response	The durable row may remain pending, and the client may not know whether bytes arrived. A request status is the prepared server outcome, not delivery proof. Restart validates/reconciles the active pointer; persistent volumes retain the audit.	Restore API health and verify the serving version. Treat pending requests as ineligible for training. If the caller must retry, issue a new request and keep its new UUID distinct from the uncertain attempt.

Failure	Current protection and observable result	Safe recovery sequence
Trainer stops or its lease expires	A stale owner cannot pass the live `retrain` lease fence at activation. The next scheduler run reconciles expired run/activation state; unresolved activation state blocks another run.	Keep the shared database and pointer intact. Allow the reconciler to query the protected activation status and inspect the resulting run. Do not delete or renew a lease manually to force candidate publication.
Activation acknowledgement or database/pointer outcome is uncertain	A durable activation ID and backend pointer permit reconciliation. An unknown outcome blocks new training; repeating a committed activation ID returns its historical result.	Query `/admin/activation/{activation_id}` with the trusted token and read current health/pointer separately. Retry status with the same activation ID if the API was briefly unavailable. Do not edit `current.json` or create another candidate until the outcome is known.
Candidate has a stale expected parent	The API compare-and-swap rejects activation (HTTP 409) without publishing against a changed parent.	Read the active version and retain the conflict record. Let the next scheduled run snapshot the current parent; do not resend stale candidate bytes with a changed parent field.
Candidate fails the gate, manifest, hash, schema, staged-load, or parity check	The run is rejected or failed and the current pointer remains on its parent. A passing gate is required for automatic activation; feature drift alone is not a bad-model detector.	Inspect the recorded run details and immutable artifact, correct the upstream cause, and wait for a new eligible snapshot. Do not manually alter a manifest or treat a same-model activation fixture as quality evidence.
A serving version is later found unsuitable	There is no automatic canary or target-error rollback detector. Rollback is an operator-initiated protected activation, limited to the previously served version or the frozen bootstrap.	Pause the scheduler and preserve evidence. Read the current version; choose the exact previous/bootstrap target; call backend `POST /admin/rollback` with `target_version`, current version as `expected_parent`, and a fresh `activation_id`. Verify the committed response, health, pointer, and a test prediction before resuming training. Never edit the pointer file directly.

These are code-level protections and recommended operator responses, not a claim that every failure in this table was injected during the Oct 5 Compose run. The recorded acceptance verifies the real rejection, activation fixture, restart persistence, and explicit rollback; it does not establish production disaster-recovery behavior.

Measurements and interpretation

The stable matched request-logging benchmark is benchmarks/logging_overhead.json. It uses fresh local one-worker API processes and isolated temporary DB/artifact/state directories; each matched on/off pair uses identical request data and cache settings.

The design has two workloads (unique and repeated), concurrency 1 and 8, three trials per case, 160 requests per trial, and logging on/off. Total: 24 cases, 3,840 requests, zero errors. Cache was enabled and dynamic batching disabled. The server and load generator shared a Windows development machine.

Workload	Concurrency	Client p50 on/off (ms)	Client p95 on/off (ms)	Throughput on/off (req/s)
Unique	1	4.24 / 2.15	7.66 / 3.25	209.17 / 436.00
Unique	8	15.91 / 10.23	24.95 / 16.89	455.27 / 729.17
Repeated	1	6.51 / 1.09	9.71 / 2.42	140.57 / 754.02
Repeated	8	36.85 / 5.01	46.45 / 22.61	208.32 / 848.63

The p50/p95 and successful-throughput values are medians across three trials. Durable audit writes increased client-visible latency in all four paired workload/concurrency combinations. The cost is especially visible for repeated inputs because cached inference is short while the two acknowledged audit writes remain. Client time covers request send through response receipt, including audit acknowledgement, HTTP, and network. The artifact does not report handler percentiles. Dashboard handler time has a server-side boundary and is not a like-for-like measure.

The recorded environment was Windows 11 build 26300, Python 3.12.14, training dataset SHA-256 1e18addf81e5e4d347cc17ee6075bbe4a42b7fa26b9e5b063e8f692a5f929d41, and native model ZIP SHA-256 4e605ea450b01531142398e273729f326df625ae21b8f0a5235f030382721db4. The Docker stack was paused during collection. These results are a local comparison, not Linux container latency, an SLO, or production capacity.

A separate private feasibility check used 32 adaptation labels with the original 1,168 fit rows and the fixed 73-row gate. RMSLE was 0.1527863 for the champion and 0.1509785 for the challenger, a 1.183% relative gain. RMSE changed from $24,598.10 to $24,510.42. Trainer/native prediction parity was exact over the gate and edge records. This retrospective calculation did not run the live request/truth/import path, write a candidate to the registry, or publish an active model. The gate is small and historically reused; this is feasibility evidence, not a claim of generalization or live promotion.

Validation and remaining checks

The exporter parity and strict split checks passed in the recorded verification. Root reported global Ruff passed. Linux images built and started under an isolated Compose project. Real HTTP traffic exercised a baseline and an unlabeled shifted stream: Neighborhood distance changed from 0.4502 to 0.5155 and exposed 40 missing and 40 unknown values; no labels were created. The browser dashboard rendered empty traffic, insufficient drift samples, a populated 60-minute view, and explicit monitoring unavailability while the stack was paused. The final browser recovery check showed a stale-data banner during API unavailability, cleared it after recovery, and displayed the active bootstrap and 40 imported labels.

The final combined suite passed 100 tests with six deprecation warnings in 39.16 seconds, and global Ruff passed. The fresh isolated Linux Compose smoke imported 40 delayed labels and produced a real rejection: champion RMSLE 0.1527862786, challenger RMSLE 0.1517689782 (0.6658% relative gain, below the 1% policy), with RMSE improving from $24,598.101 to $24,548.884. The controlled same-model fixture then committed the activation path. Exact request UUID and captured model version, imported source and observation IDs, and serving pointer matched before and after backend restart. A separate check rolled back the fixture to the frozen bootstrap; repeating the same activation ID returned idempotent committed status, and an activation with a stale expected parent was rejected with HTTP 409. These checks validate serving transitions; the fixture and rollback make no accuracy claim. The final UI recheck also confirmed the recovered dashboard cleared its stale-data banner and displayed the bootstrap with 40 imported labels; the form's API sample selected `Sawyer` and returned a $143,950 prediction.

The final audit ran once after model selection on the frozen bootstrap, with no role in model selection. On 73 rows it recorded RMSLE 0.1385910503, MAE $16,402.36, and RMSE $24,117.92. The partition was part of the old 292-row holdout, so this is descriptive evidence rather than an untouched external accuracy estimate. Full raw run, audit, rollback, environment, and monitoring artifacts are indexed in report/evidence. Remote GitHub Actions was not run. The Compose demo flow was exercised; no presentation is claimed.

Sources and related files

- README.md - setup and operational commands
- DESIGN.md - dated design decisions and measured tradeoffs
- PLAN.md - authorized scope and current completion boundary
- Demo runbook - steps and expected evidence; no presentation claim
- Phase 3 report - native inference and earlier optimization benchmarks
- Split manifest and drift reference
- Logging benchmark artifact
- Repository